Date: .... No: ....

Problem 1: 2.5
Problem 2: 2.5
Problem 3: 2.5
Problem 4: 2.5
9,85

Tên: Vũ Lê Thanh Phát
MSSV: N25DECE062 Tờ 1:

Problem 1: Singly Linked List - In-Place Reversal & Pointer Tracking
Given a singly linked list head -> 10 -> 25 -> 40 -> 15 -> NULL

Task 1: Write a function/algorithm in pseudocode to reverse the singly linked list including, three pointers (prev, curr, next) to complete your function

Task 2: Draw a diagram or step-by-step pointer trace table showing the ~~positi~~ positions and values of the 3 three pointers (prev, curr, next) after each iteration until the list is fully reversed

Problem 2: Circular Linked List - Insert at Head
Given a singly circular linked list head -> 10 -> 20 -> 30 -> (points back to head)

Task 1: Write pseudocode for the function InsertHead(head, newValue) to insert a new node containing newValue at the front of the circular linked list while maintaining its circular structure

Task 2: Trace the execution steps (via text description or pointer diagrams) when inserting newValue = 5 into given list.

Problem 3: Doubly Linked List - Insert Before Target & Edge Case
Given a doubly linked list containing integers

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

Task 1: Write psudocode for InsertBefore(head, target, new value) to insert a new node containg newValue immediatly before the first occurance of target

Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leaks or broken links

Task 3: Explicitly handle the following two edge cases in code/psuedo code:
1. target is located at the head node (head
2. target does not exist in the list

Problem 2: Sorted Linked List - In order Insertion
Given a singly linked list sorted a ascending order head
head -> 2 -> 5 -> 8 -> NULL

Task 1: Write the function SortedInset(head, val) to insert val into the list such the the ascending order is preserved

Task 2: Construct a dep-by-step trace table detailing pointer iterations and condition evaluation when inserting val = 6 into the list

Solution

2.5/5 Problem 1:

1.5/5 Psuedo Code

```
Reverse(head):
    prev = NULL
    curr = head
    next = NULL
    while curr ≠ NULL:
```

Date: .... No: ....

```
        next = curr.next
        curr.next = prev
        prev = curr
        curr = next
    return prev
```

1.25 Task 2

Trace table (10 -> 25 -> 40 -> 15)

Iter	next	After reverse	prev	curr
Init	-	~	NULL	10
1	25	10 -> NULL	10	25
2	40	25 -> 10	25	40
3	15	40 -> 25	40	15
4	NULL	15 -> 40	15	NULL

Result: head -> 15 -> 40 -> 25 -> 10 - NULL

2.5 Problem 2.

(2.5) Insert Head (head, NewValue):

```
    node = new Node (newValue)
    if head == NULL
        node.next = node
        return node
    tail = head
    while (tail.next ≠ head)
        tail = tail.next
```

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

node.next = head
tail.next = node
return node

1.25 Trace NewValue = 5

1. Create node 5
2. Find tail : 10 -> 20 -> 30 : stop bc 30.next == head
3. node.next = 10
4. 30.next = node(5)
5. head = node(5)

Result 5 -> 10 -> 20 -> 30 ->

2.5 Problem 3:

~~Insert before~~

InsertBefore(head, target, NewValue):
    cur = head
    while (cur ≠ NULL and cur.data ≠ target):
        cur = cur.next
    If cur == NULL:
        return head
    node = new Node(newValue)
    node.prev = cur.prev
    node.next = cur

Date: .... No: ....

Ten: Vũ Lê Thái Hải

MSSV: M25 DECE067 Tờ 2

```
if cur = NULL
    return head
node = ne
if cur.prev ≠ NULL
    cur.prev.next = node
else:
    head = node
cur.prev = node
return head
```

Assignment order 1 -> 2 -> 3 -> 4

1 and 2 come first: the new node is fully wired without touching the exist list.

3 must come before 4, bc 4 overwrites cur.prev and we would lose access to the prev one node.

This way no link break and no node is leak

2.5 Problem 4:

```
SortedInsert (head, val)
    node = new Node (val)
    if (head = NULL case or val ≤ head.data:
        node.next = head
        return node
```

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

```
cur = head
while (cur.next ≠ NULL and cur.next.data < val:)
    cur = cur.next
node.next = cur.next
cur.next = node
return head
```

Trace with val = 6 (2 -> 5 -> 8)

Step	cur	cur.next.data	< 6 ?	Action
Start	-	head = 2	6 < 2 ? No	Skip
1	2	5	Yes	cur = 5
2	5	8	No	Stop
Insert	5	-	-	node.next = 8
				5.next = 6

Result 2 -> 5 -> 6 -> 8 -> NULL